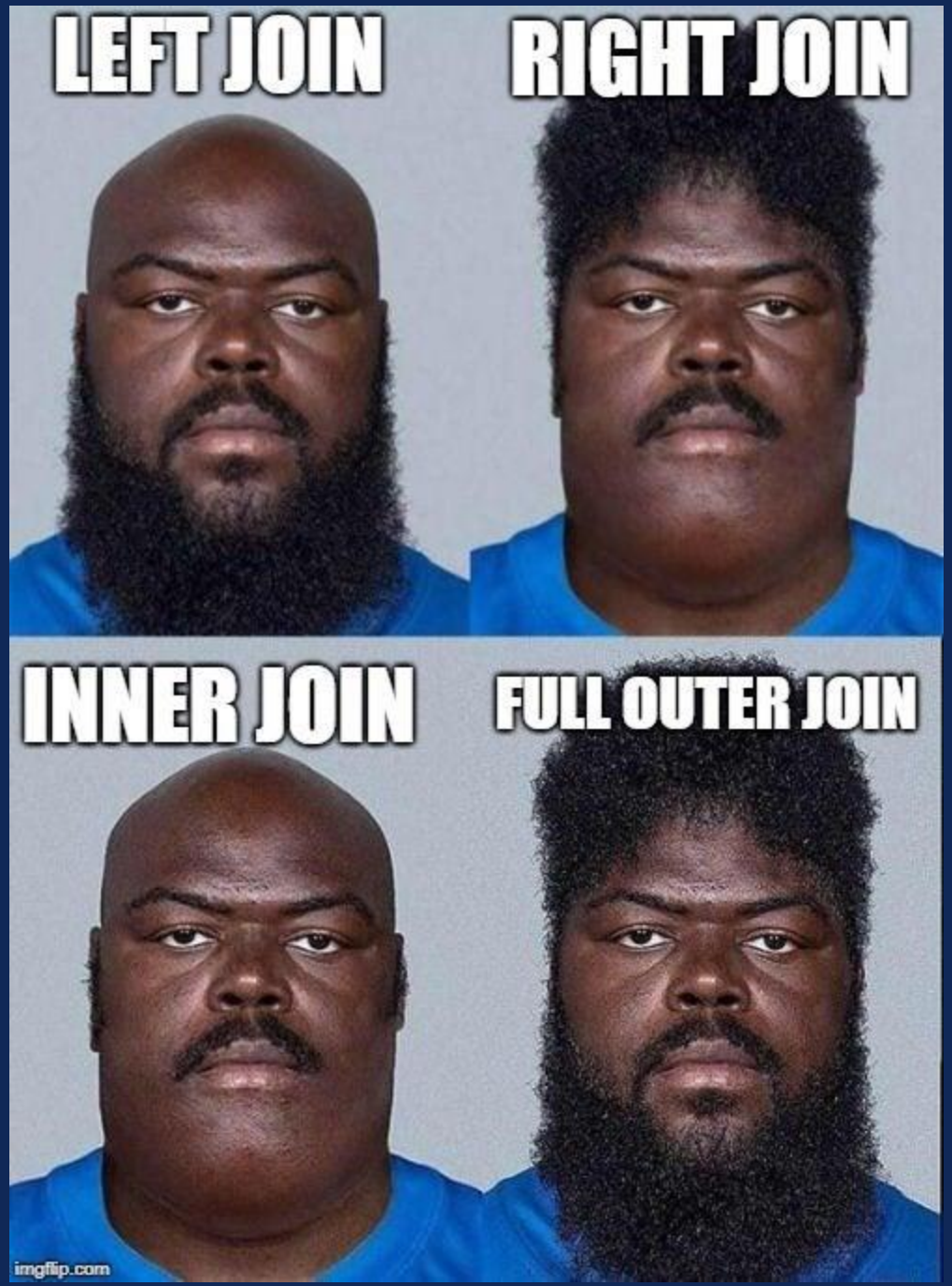
JOINING & TRANSFORMING DATA

Garrett Bullock, PT, DPT, DPhil


LEFT JOIN
RIGHT JOIN
INNER JOIN
FULL OUTER JOIN
imgflip.com

TYPES OF DATA FRAME FORMATS

Wide Data

- Each subject has one row
- Different measurement time points or conditions are separate columns
- Much more human readable

Long Data

- Each row is a single observation
- Each time point or condition is a different row
- Computer readable

Wide Format

Team	Points	Assists	Rebounds
A	88	12	22
B	91	17	28
C	99	24	30
D	94	28	31

Long Format

Team	Variable	Value
A	Points	88
A	Assists	12
A	Rebounds	22
B	Points	91
B	Assists	17
B	Rebounds	28
C	Points	99
C	Assists	24
C	Rebounds	30
D	Points	94
D	Assists	28
D	Rebounds	31

Joining & Transforming Data Is Mostly Done with dplyr

- All about pipes: %>%
- dplyr's verbs are organized into four groups
- based on what they operate:
  - Rows
  - Columns
  - Groups
  - Tables.

Joining & Transforming Data Is Mostly Done with dplyr

- All about pipes: %>%
- dplyr's verbs are organized into four groups
- based on what they operate:
  - Rows
  - Columns
  - Groups
  - Tables.

Rows

filter() function: changes which rows are present without changing their order.

arrange() : which changes the order of the rows without changing which are present.

distinct() : gives on exact match per row

Columns are left unchanged

Rows: Filter

- Allows you to keep rows based on the values of the columns

- First argument = data frame
- Second and subsequent arguments = conditions that must be true to keep the row.

Example of code:

```
data  %>%
        filter(height_cm > 165)
```

Rows: Filter

- Allows you to keep rows based on the values of the columns

- First argument = data frame
- Second and subsequent arguments = conditions that must be true to keep the row.

- Example of code:

```
data  %>%
      filter(height_cm > 165)
```

- Other operations: < ; >= ; <= ; == ; |

There's a useful shortcut when you're combining | and ==: %in%. It keeps rows where the variable equals one of the values on the right.

Rows: Filter: Useful Shortcut

- When combining | and ==:
- %in%.
- Keeps rows where the variable equals one of the values on the right
- Will be used in the practical!

Rows: Arrange

- Changes the order of the rows based on the value of the columns.

- If you provide more than one column name, each additional column will be used to break ties in the values of the preceding columns.

- Example of code:

```
data  %>%
      arrange(height_cm)
```

- To flip the order

```
data  %>%
      arrange(desc(height_cm))
```

Rows: Distinct

- Works on rows.
- Provides one exact match for the combination of variables within rows
- Example of code:

```
data  %>%
      distinct(height_cm) #Keeps only the one variable (column)
```

- Keep all columns

```
data  %>%
      distinct(height_cm, .keep_all = TRUE)
```

Columns

- Verbs that affect the columns without changing the rows.
- mutate() creates new columns from other columns
- select() changes which columns are present
- rename() changes the names of the columns
- relocate() changes the positions of the columns.
- Row are left unchanged

Columns: Mutate

- mutate() creates new columns from other columns

- Example code:

```
data %>%
        mutate(height_in = height_cm  / 2.54)
```

Columns: Select

- changes which columns are present
- Example code:

```
data %>%
        select(record_id, height_cm )
```

- Can also un-select variables

```
data %>%
        select(-record_id, -height_cm )
```

- Can also select through a range of variables

```
data %>%
        select(record_id : outcome)
```

Columns: Rename

- changes the names of the columns

- Variable on left, new name
- Variable on right, old name

- Example code:

```
data %>%
        rename(height_centimeters = height_cm )
```

Columns: Rename: Useful Shortcut

- If you have a bunch of variables that need changing
- Use the janitor package
- With clean_names() function
- Example code:

```
data <- data %>%
        clean_names( )
```

Columns: Relocate

- changes the positions of the columns.
- Moves the variable on the left
- Default is to the front of the variable in the right of the expression
- Can be specific before or after
- Example code:

```
data %>%
        relocate(height_cm, record_id )
```

- If you want it after

```
data %>%
        relocate(height_cm, .after = record_id )
```

WHERE THE REAL POWER COMES IN

WHERE THE REAL POWER COMES IN

```
new_data <- data %>%
            filter(height_cm > 165) %>%
            mutate(height_in = height_cm / 2.54) %>%
            select(record_id, height_in, height_cm, outcome) %>%
            arrange(desc(height_cm))
```

Groups

- Allows grouping by different commands
- group_by() : Groups by different logic (left v right; male v female)
- summarize() : Aggregates by different commands (mean, median, etc.)
- slice() : Extract different rows from each group
- Row AND Columns are left unchanged

Groups: Group By

- Groups by different logic (left v right; male v female)

```
data %>%
        group_by(hand_dominance)
```

Groups: Summarise

- Aggregates by a function

```
data %>%
        summarise(mean_height_cm = mean(height_cm, na.rm = T)
```

Groups: Slice

- Extracts different rows by groups

```
data %>%
        slice_head(n = 1)
```

Joins

- Very rarely will you work on only one dataset
- Will nee to combine two or more
- This requires specific information to be able to join data
- This is also known as cross-walking data

Joins: Keys

- Two types of keys

- Primary Key: One or more datapoints that identify unique observations
  - Name: Mark Buehrle
  - Date: 2014-01-01
  - Unique ID: 1234567
  - Car Model: HA1721

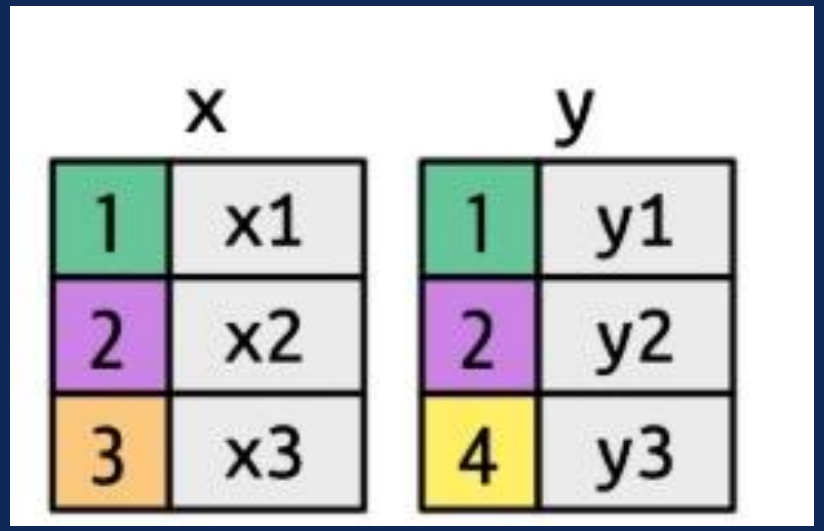
- Foreign Key: One or more datapoints that connect unique observations between data frames
  - These specific key(s) will be the same between data frames

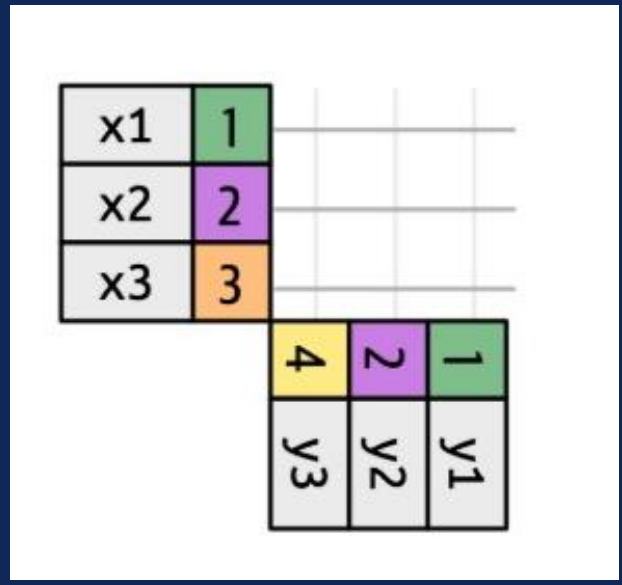
Joins: The Key are the Keys!

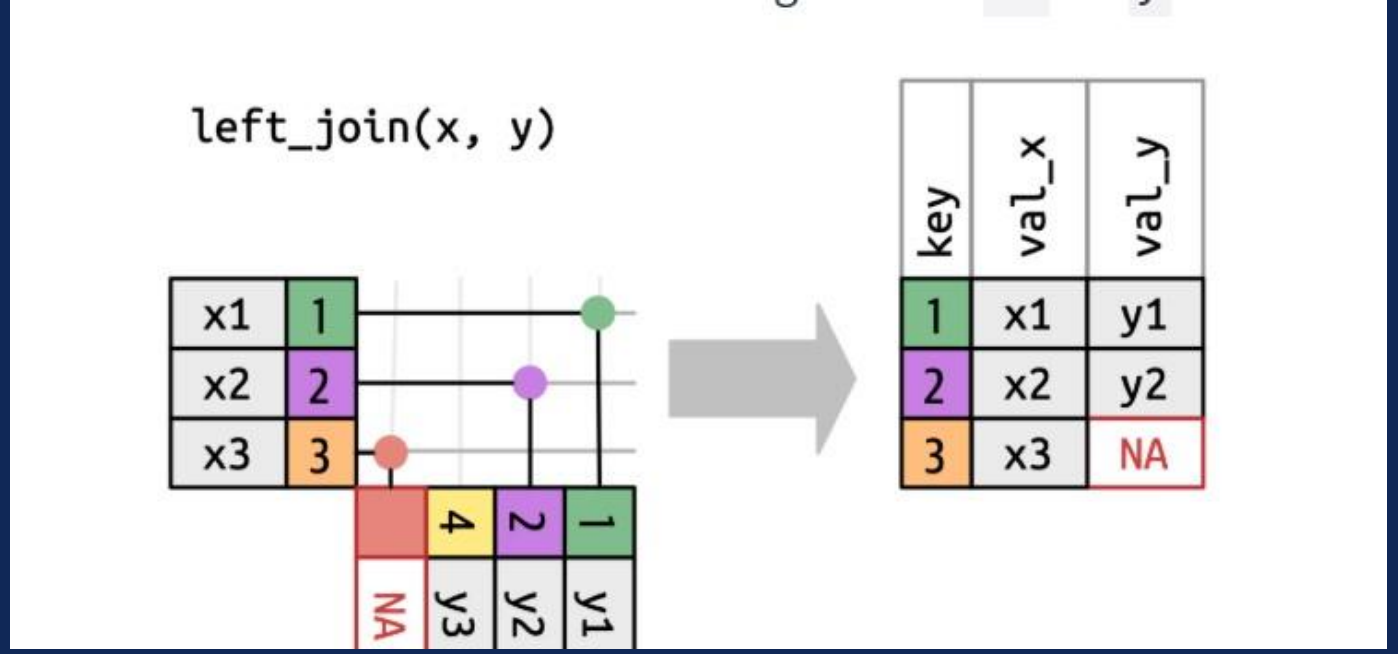
- The primary and foreign keys should align
  - If there are missing data, these cannot be joined

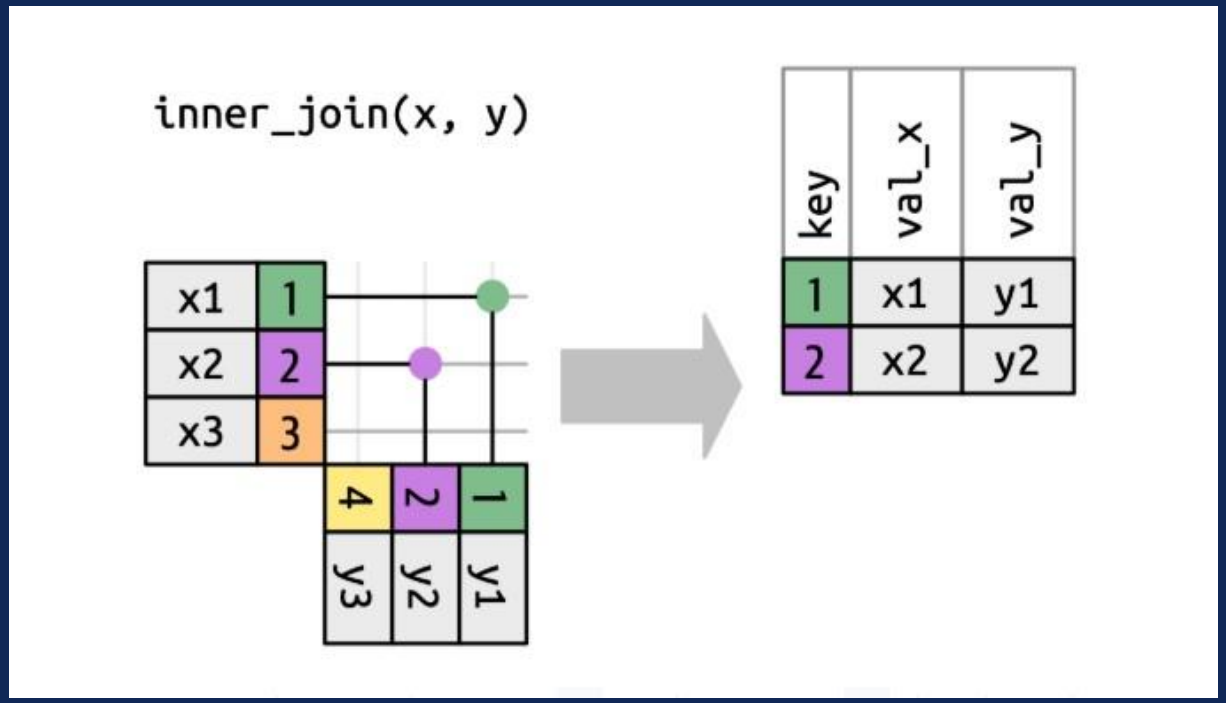
- Will join on the keys

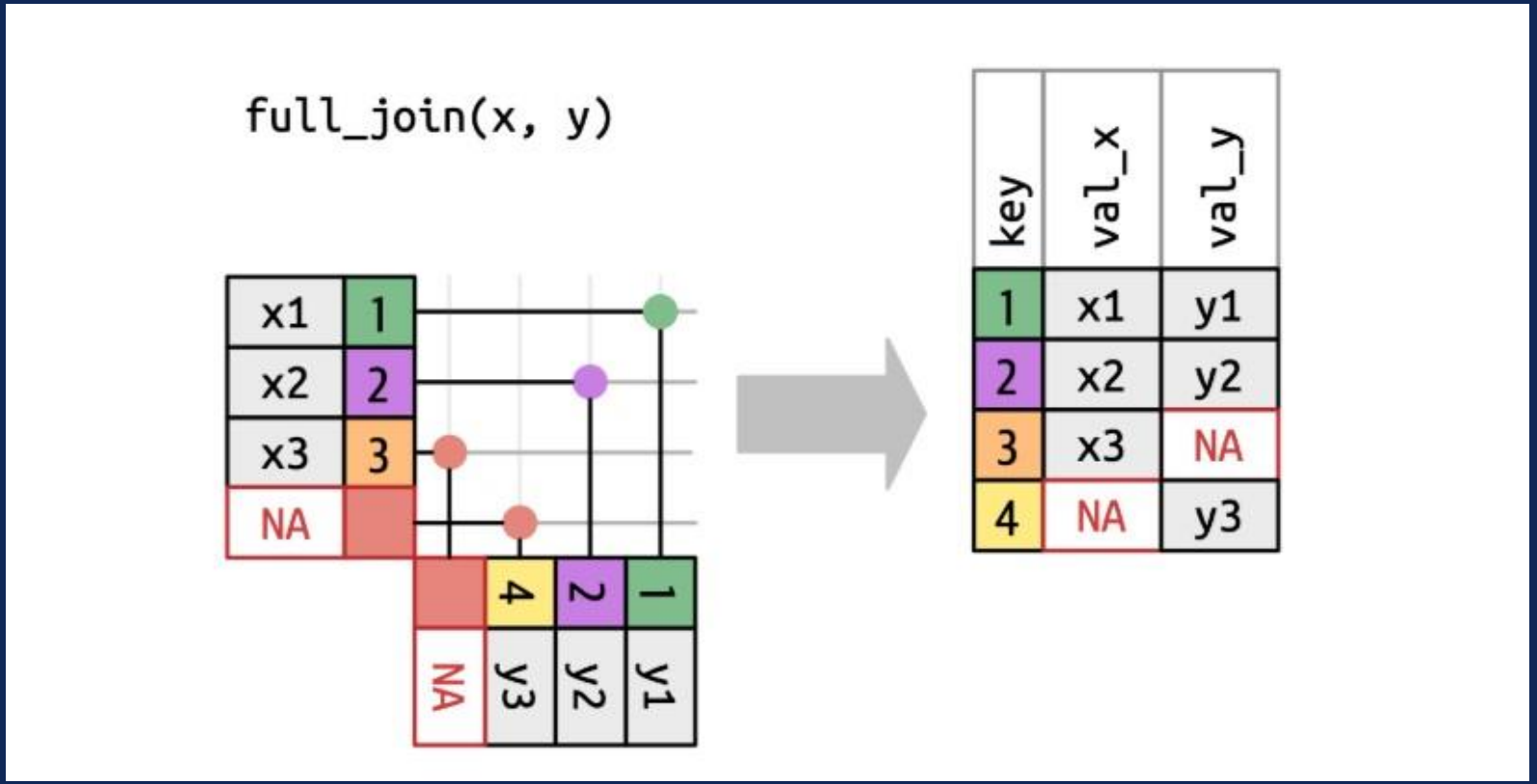
- There are 6 major types of join
- left_join():The data frame on the left will have the same # rows
- inner_join() : Will only join data that is in BOTH
- right_join() : The data frame on the right will have the same # rows
- full_join() : All rows from BOTH data frames are included
- semi_join() : Keeps only rows that are in the left with a match in right
- anti_join() : Row that are NOT in BOTH data frames are included

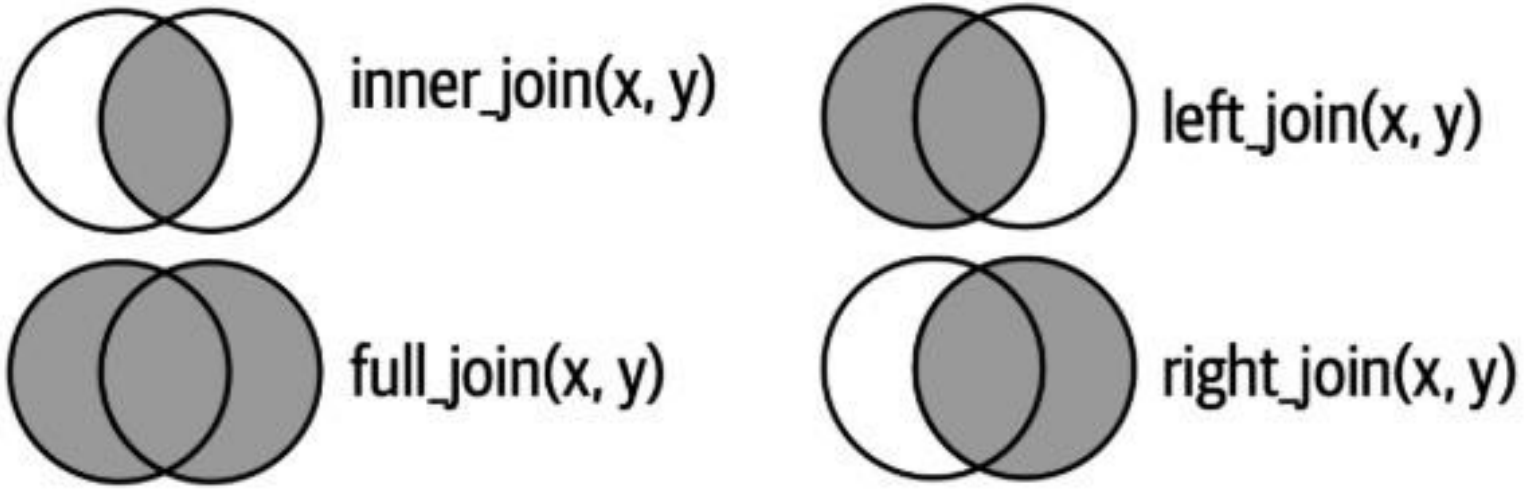

x
1
x1
2
x2
3
x3
y
1
y1
2
y2
4
y3


x1
1
x2
2
x3
3
4
y3
2
y2
1
y1


left_join(x, y)
x1
x2
x3
1
2
3
4
2
1
NA
y3
y2
y1
key
val_x
val_y
1
x1
y1
2
x2
y2
3
x3
NA


inner_join(x, y)
x1
x2
x3
1
2
3
4
2
1
y3
y2
y1
key
val_x
val_y
1
x1
y1
2
x2
y2


full_join(x, y)
x1
x2
x3
NA
1
2
3
4
2
1
NA
y3
y2
y1
key
val_x
val_y
1
x1
y1
2
x2
y2
3
x3
NA
4
NA
y3


inner_join(x, y)
left_join(x, y)
full_join(x, y)
right_join(x, y)

Joins: What You Use in Reality

- left_join():The data frame on the left will have the same # rows
- inner_join() : Will only join data that is in BOTH
- full_join() : All rows from BOTH data frames are included
- anti_join() : Row that are NOT in BOTH data frames are included

- Almost 90% of the time: will use left_join():

Joins: Example

- Tasked to evaluate Emergency Department admissions for patients that originally were seen within the orthopaedic surgery department 30 days post, all patients in the past year.

- Have data from multiple data streams:
  - Orthopaedic Surgery
  - Emergency Department

- Need to combine these data to evaluate this question.

Joins: Example

Data You Need to Evaluate This Important Clinical Question:

- Orthopaedic Surgery
  - Date, Medical Number, Visit Type
- Emergency Department
  - Date, Medical Number, Admission to Hospital (y/n)

Joins: Example

Threats to the Data:

- Emergency Department will have much more data than people that have seen an orthopaedic surgeon: All comers
- Emergency Department data will include admissions past the 30 day post orthopaedic surgery window

Joins: Example

- Must Join on the Keys!
- Key: Medical Number (mrn)

- Example Code:

- Ortho_emergency_joined_data <- left_join(ortho, emergency, join_by(mrn))

Why join this way? Remember this is a left join…

Joins: Example

Example Code:

- Ortho_emergency_joined_data <- left_join(ortho, emergency, join_by(mrn))

- Joining with emergency left, would include all patients, and not just ortho
  - Why not include date?

  - Because we want a 30-day window
  - Probably not on the same exact same date for both

Final Thoughts

- Grouping, Slicing, and Joining data are part of daily data science life
- The Key are the Keys